1.4 Data frames

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 14 October 2026, 19:18

This unit introduces data frames in R, covering their creation, subsetting and filtering (with both base R and the tidyverse), and includes practice exercises on accessing and manipulating structured linguistic data.

Table of contents

Preparation . . . 1
Recommended reading . . . 1
Word frequencies II . . . 2
Essential R concepts . . . 3
Filtering . . . 4
This looks unintuitive – is there another way to filter in R? . . . 5
Exercises . . . 7
Tier 1 . . . 7
Tier 2 . . . 8
Tier 3 . . . 8
References . . . 11

Preparation

> Script
>
> You can find the full R script associated with this unit here.

Recommended reading

Baayen (2008): Chapter 1

Winter (2020): Chapter 1.10–1.16

Suggested video tutorial:

Using the Data Frame in R (DataCamp; 5 min)

Learn How to Subset, Extend & Sort Data Frames in R (DataCamp; 7 min)

Word frequencies II

Recall our simple linguistic dataset from the previous unit:

Table 1

Lemma	Frequency
start	418
enjoy	139
begin	337
help	281

In the previous unit, we treated the columns as one-dimensional, indexed sequences of elements, i.e., vectors:

```
lemma <- c("start", "enjoy", "begin", "help")

frequency <- c(418, 139, 337, 281)
```

In fact, R allows us to combine these two vectors into an actual spreadsheet. To this end, we need to apply the `data.frame()` function to two vectors of our choice. Note that they need to have the same length:

```
data <- data.frame(lemma, frequency)

print(data)
```

```
  lemma frequency
1 start       418
2 enjoy       139
3 begin       337
4  help       281
```

Essential R concepts

The variable `data` is no longer a vector but a **data frame** (often abbreviated as '**df**'). Once again, each element carries its own label and can therefore be accessed or manipulated.

Since data frames are two-dimensional objects, the **subsetting** notation in square brackets `[ ]` needs to reflect that. This is the general pattern:

$$\text{df[row, column]} \tag{1}$$

Say we're looking for the element at the intersection of the first row and first column. Applying the pattern above, we can access it like so:

```
data[1,1]
```

```
[1] "start"
```

But what if we needed the entire first row? We'd simply omit the column part. Note, however, that the comma `,` needs to remain:

```
data[1,]
```

```
  lemma frequency
1 start       418
```

When subsetting by columns, we can use either the square bracket notation `[ ]` or the **column operator** `$`:

```
data[,1]
```

```
[1] "start" "enjoy" "begin" "help"
```

```
data$lemma
```

```
[1] "start" "enjoy" "begin" "help"
```

Filtering

Not all the observations contained in a data frame are necessarily relevant for our research. In such cases, it may be important to subset the rows and columns according to certain criteria.

Assume we only need those observations where the `lemma` frequencies are greater than 300. We can filter the dataset accordingly by specifying

1. the data frame,
2. the column of interest, and
3. the condition to apply to the rows.

You can read the code below as

> 'Take the data frame `data` and subset it according to the column `data$frequency`. Show me those rows where the values of `data$frequency` are greater than 300.'

```
data[data$frequency > 300, ]
```

```
  lemma frequency
1 start       418
3 begin       337
```

What if we wanted to filter by `lemma` instead? To make it more concrete, assume we're looking for frequency data on the verbs *start* and *help* (but not *begin* and *enjoy*).

Let's first access the row containing data on *start*:

```
data[data$lemma == "start", ]
```

```
  lemma frequency
1 start       418
```

Next, we add a second, analogous condition. Combining multiple statements requires a **logical operator**. In this code chunk, we're using `|`, which corresponds to a logical 'or' (also known as a 'disjunction').

```
data[data$lemma == "start" | data$lemma == "help", ]
```

```
  lemma frequency
1 start       418
4  help       281
```

Why do we need to use 'or' (|) and not 'and' (&)?

The idea of combining statements somewhat naturally suggests a conjunction, which could be achieved via `&`. How come R doesn't return anything if we do it that way?

```
data[data$lemma == "start" & data$lemma == "help", ]
```

```
[1] lemma     frequency
<0 rows> (or 0-length row.names)
```

This looks unintuitive – is there another way to filter in R?

Yes, absolutely. The callouts below demonstrate a few popular alternatives. In the end, the exact way you filter doesn't really matter, so long as you (as well as the people who have to work with your script) can understand what you're trying to achieve with your code. Always make sure to add comments to your filtering operations!

`subset()`

Almost every subsetting operation we perform with square brackets can also be performed using the `subset()` function. Here are some expressions that are equivalent to the ones above:

```
subset(data, frequency > 300)
```

```
  lemma frequency
1 start       418
3 begin       337
```

```
subset(data, lemma == "start" | lemma == "help")
```

```
  lemma frequency
1 start       418
4  help       281
```

`tidyverse`

The `tidyverse` ecosystem is a collection of packages specifically designed for handling typical data science tasks as comfortably and elegantly as possible, supplying countless helper functions for data manipulation, transformation and visualisation. Installation instructions are provided in 1.5 Libraries.

An extensive guide to the main `tidyverse` functions is provided in Chapter 3 of the free eBook R for Data Science (2nd edition). Because of its clarity, `tidyverse` syntax is used for most of the more advanced code in this reader.
Let's generate a `tidyverse`-style data frame, the **tibble**:

```
library(tidyverse)

data2 <- tibble(
  lemma = c("start", "enjoy", "begin", "help"),
  frequency = c(418, 139, 337, 281)
)

print(data2)
```

```
# A tibble: 4 x 2
  lemma frequency
  <chr>     <dbl>
1 start       418
2 enjoy       139
3 begin       337
4 help        281
```

We can single out certain columns with `select()`:

```
select(data2, lemma)
```

```
# A tibble: 4 x 1
  lemma
  <chr>
1 start
2 enjoy
3 begin
4 help
```

It is very easy to filter the data frame according to certain criteria:

```
filter(data2, frequency > 300)
```

```
# A tibble: 2 x 2
  lemma frequency
  <chr>     <dbl>
1 start       418
2 begin       337
```

```
filter(data2, lemma == "start" | lemma == "help")
```

```
# A tibble: 2 x 2
  lemma frequency
  <chr>     <dbl>
1 start       418
2 help        281
```

The `tidyverse` features a special **pipe** operator `%>%` that can be used to pass the output of one function on to the next one. It is conceptually similar to the coordinating conjunction *and*. The code can be rewritten in pipe notation as follows:

```
# Read as: "Take data2 and select the column with the name 'lemma'."
data2 %>%
  select(lemma)

# Read as: "Take data2 and show me those rows where frequency is
 ↪  greater than 300."
data2 %>%
  filter(frequency > 300)

# Read as: "Take data2 and show me those rows that correspond to the
 ↪  lemma 'start' or 'help'."
data2 %>%
  filter(lemma == "start" | lemma == "help")
```

Exercises

Solutions

You can find the solutions to the exercises here.

Tier 1

Exercise 1. Print the following elements by subsetting the data frame `data` accordingly.

- `337`
- `begin`
- `enjoy`
- `enjoy 139`

- the entire `frequency` column

Exercise 2. Consider the following R code:

```
data[data$frequency < 150 | data$frequency > 400,]
```

a. Which rows of the data frame `data` will this code return? Describe which observations will be included.
b. Explain in words what each part of the code does.
c. Run the code in R and compare the actual output with your prediction. Was your prediction correct? If not, where did your reasoning go wrong?

Tier 2

Exercise 3 (Extension of Ex. 3 from Vectors)**.** Check whether the following verbs are represented in the `lemma` column: *enjoy*, *hit*, *find*, *begin*. If they are, print their frequency information.

Exercise 4 (Extension of Ex. 4 from Vectors)**.** Use `which()` to find the rows where the frequency is greater than 200, and then print the lemma and frequency of those rows only.

Tier 3

Exercise 5. Diachronic corpora comprise data on language use across different time periods. This data frame indicates the frequencies of certain modal verbs across two time periods:

```
modals_df <- data.frame(
  modal = c("can", "could", "may", "might", "must", "shall", "should",
   ↪  "will", "would"),
  period1 = c(128, 68, 55, 21, 44, 19, 35, 85, 97),
  period2 = c(142, 83, 41, 30, 39, 12, 52, 94, 119)
)
```

- Print the rows with the most and least frequent modal verb for each time period.

  > **Hint**
  >
  > The functions `min()` and `max()` return the minimum and maximum value of a numeric vector, respectively.

- Calculate the percentage change in frequency for each modal verb between `period1` and `period2`.

> **Hint**
>
> The percentage change in word frequency $f(w_i)$ from Period 1 to Period 2 can be obtained as follows:
>
> $$\frac{f_{\text{Period 2}}(w_i) - f_{\text{Period 1}}(w_i)}{f_{\text{Period 1}}(w_i)} \times 100$$

- Create a new column `trend` with the values `"increasing"` and `"decreasing"` based on whether the frequency increased or decreased between the two periods.

> **Hint 1**
>
> This can be achieved without invoking control structures (e.g., `if`-statements). The following code defines a `"trend"` column for `modals_df`, but assigns the same value to all verbs (`"increasing"`).
>
> ```
> modals_df$trend <- "increasing"
> ```
>
> Using subsetting operations, identify the rows with a negative percentage change, and reassign their `trend` values accordingly.

> **Hint 2**
>
> It is possible to subset a data frame and access a specific column vector at the same time:
>
> ```
> modals_df[modals_df$modal == "can",]$period1
> ```
>
> ```
> [1] 128
> ```

Exercise 6 (Extension of Ex. 8 from Vectors)**.** Write a function that performs part-of-speech (POS) annotation on the sentence *The quick brown fox jumps over the lazy dog.* Here are a few code snippets to help you get started:

- You can split sentences into tokens using `tokenize_words()` from the `tokenizers` package.

```
library(tokenizers)
library(tidyverse)

text <- "Colorless green ideas sleep furiously."
text_tokenized <- tokenize_words(text)

# Convert the tokens to lower case
tokens_lower <- tolower(text_tokenized[[1]])
```

- Vectors can have name attributes:

```
word <- "read"

# Give it a name
names(word) <- "verb"

# Get rid of its name
word <- unname(word)
```

- There are several ways to apply conditional logic:

```
things <- c("apple", "cherry", "pear", "cucumber", "coconut")
fruits <- c("apple", "cherry", "pear")
vegetables <- c("cabbage", "carrot", "cucumber")

# Base R
food_analysis <- ifelse(things %in% fruits, "fruit", "not_fruit")

# Tidyverse
food_analysis2 <- case_when(
  things %in% fruits ~ "yes", # if elements from "things" are in "fruits",
 ↪  return "yes", else
  TRUE ~ "no"                 # return "no" (default)
)
```

- If several conditions need to be checked, `ifelse()` statements have to be nested, whereas `case_when()` simply checks the cases in order:

```
# Base R
complex_food_analysis <- ifelse(things %in% fruits, "fruit",
       ifelse(things %in% vegetables, "vegetable",
              "unknown"))

# Tidyverse
complex_food_analysis2 <- case_when(
  things %in% fruits ~ "fruit", # if elements from "things" are in
 ↪  "fruits", return "fruit", else
  things %in% vegetables ~ "vegetable", # if they're in "vegetables",
 ↪  return "vegetable", else
  TRUE ~ "unknown"     # return "unknown"
)
```

References

Baayen, R. Harald. 2008. *Analyzing Linguistic Data: A Practical Introduction to Statistics Using r*. Camrbidge: Cambridge University Press.

Winter, Bodo. 2020. *Statistics for Linguists: An Introduction Using r*. New York; London: Routledge.